

REPUBLIQUE DU CAMEROUN
PAIX-TRAVAIL-PATRIE
MINISTERE DE L'ENSEIGNEMENT
SUPERIEUR
UNIVERSITE DE BAMENDA



REPUBLIC OF CAMEROON
PEACE-WORK-FATHERLAND
MINISTRY OF HIGHER
EDUCATION
UNIVERSITY OF BAMENDA



INSTITUT DES SCIENCES ECONOMIQUES ET INFORMATIQUE DE GESTION

AUT N° 05/2002/MINESUP du 03 Janvier
2005/2006/2011/2014/2015

AVANT PROJET DE MÉMOIRE

SPECIALITE : GENIE LOGICIEL ET MANAGEMENT DES DONNEES

MISE EN ŒUVRE D'UN OUTIL D'AIDE A LA MIGRATION POUR LES ENVIRONNEMENTS CONTENEURISES ET LEURS ADMINISTRATIONS

Rédigé et soutenu par :

MOHAMADOU DAHIROU

Matricule : 241GLO047M1

Sous l'encadrement de :

M. NGANOMBEL GABIN

Table des matières

I. CONTEXTE ET JUSTIFICATION	3
II. PROBLEMATIQUE ACTUELLE	5
➤ L'érosion de la reproductibilité : Les serveurs « Flocons de Neige »	5
➤ La "Matrice de l'Enfer" : Conflits et pollutions système.....	5
➤ L'opacité des interdépendances : Une architecture « boîte noire ».....	6
➤ Problèmes de portabilité entre environnements (développement, test, production).....	6
➤ Scalabilité coûteuse (ajout de ressources matérielles).....	6
➤ Temps de déploiement longs pour les nouvelles versions.....	6
➤ Le problème d'administration	6
➤ Problème d'optimisation des ressources utilisés.....	6
III. QUESTIONS DE RECHERCHE	7
IV. OBJECTIFS.....	7
V. PROBLEMES OBSERVES ET LES HYPOTHESES DE L'ETUDE	8
➤ Problèmes Observés.....	8
➤ Hypothèses (Gains Attendus).....	9
VI. INTERET DE L'ETUDE.....	10
➤ Intérêt technologique et méthodologique.....	11
➤ Intérêt économique et Opérationnel	12
➤ Intérêt Stratégique et Pédagogique	12
VII. APPROCHE METHODOLOGIQUE	13
➤ Approche de recherche choisie : Étude de cas comparative et expérimentation	14
➤ Outils et Technologies Utilisés	15
➤ Outil d'Aide à la Migration Automatisée :.....	16
VIII. SCOPE ET LIMITES DE L'ETUDE	17
➤ Scope de l'Étude	17
➤ Limites de l'Étude	19

I. CONTEXTE ET JUSTIFICATION

À l'ère du Cloud Computing et des architectures microservices, la rapidité et la fiabilité du déploiement sont devenues des enjeux stratégiques pour l'ingénierie logicielle. Pourtant, de nombreuses organisations se heurtent encore aux limites du **déploiement classique** dit « Bare Metal » ou installation directe sur l'hôte une approche qui, bien qu'historique, s'avère aujourd'hui inadaptée à la complexité des applications modernes.

En effet, Le déploiement et la gestion d'applications ont été profondément transformés par l'avènement de la **conteneurisation** (notamment avec **Docker**) et de l'**orchestration** (avec **Kubernetes**). Ces technologies offrent des avantages indéniables en terme de portabilité, de scalabilité, d'isolation et d'efficacité des ressources

Cependant, le passage des applications web existantes, souvent conçues avec des architectures monolithiques et des dépendances complexes, vers cet environnement conteneurisé représente un défi significatif. Ce processus est généralement manuel, long, sujet aux erreurs et nécessite une expertise pointue. C'est ici qu'intervient l'idée d'un **outil d'aide automatisé**, capable de rationaliser et d'accélérer cette transition. L'objectif de ce mémoire est d'analyser en profondeur les concepts sous-jacents à l'utilisation et au développement de tels outils

Bien qu'il existe des outils comme « Docker » pour assurer cette migration et malgré les avantages indéniables offerts par la conteneurisation, ces entreprises peinent à effectuer cette transition par manque d'expertise DevOps.

Le choix de ce thème se justifie pour plusieurs raisons à savoir :

- **Actualité technologique** : La conteneurisation est au cœur des architectures logicielles modernes (DevOps, Cloud-Native).
- **Demande du marché** : Les entreprises cherchent activement des experts capables de gérer la transition vers des environnements conteneurisés.

- **Complexité et richesse :** Le sujet est suffisamment vaste pour permettre une exploration approfondie de plusieurs facettes (technique, sécurité, organisationnel).

C'est dans cet ordre d'idée que nous nous sommes intéressés à informatiser une solution pour permettre à toutes les structures souhaitant accélérer la transition vers des architectures modernes sans nécessité de compétences DevOps.

II. PROBLEMATIQUE ACTUELLE

Au Cameroun, la migration des applications web existantes demeure un défi majeur pour de nombreuses entités publiques, parapubliques et privées. Une grande partie du patrimoine applicatif est constituée d'applications web développées sur des architectures plus traditionnelles (monolithique ou 3-tiers), souvent déployées directement sur des serveurs physiques ou des machines virtuelles, avec des dépendances complexes et des configurations spécifiques à leur environnement d'origine.

En effet, ces infrastructures traditionnelles présentent plusieurs limitations à savoir :

➤ **L'érosion de la reproductibilité : Les serveurs « Flocons de Neige »**

Le premier obstacle majeur est l'accumulation de configurations manuelles. Comme le souligne **Martin Fowler**, un serveur configuré directement sur l'hôte finit par diverger de son état initial. Chaque mise à jour de sécurité, chaque modification de variable d'environnement ou installation de patch crée une dérive.

- ✚ **La conséquence** : Il devient impossible de garantir que l'environnement de développement est identique à celui de production.
- ✚ **Le risque** : Le fameux syndrome du « *ça marche sur ma machine* ». En cas de panne, la reconstruction d'un serveur à l'identique prend des heures, voire des jours, car la « **recette** » de cuisine du serveur n'est jamais intégralement documentée.

➤ **La "Matrice de l'Enfer" : Conflits et pollutions système**

Dans un environnement classique, toutes les applications partagent les bibliothèques du système d'exploitation (/usr/lib, System32). **Solomon Hykes** (2013) a mis en évidence le cauchemar logistique que cela représente.

- ✚ **Le conflit de versions (Problème de la Cohabitation)** : Si l'Application A requiert la bibliothèque lib-ssl v1.1 et que l'Application B nécessite la v3.0, l'administrateur système est face à une impasse. L'installation de l'une peut rendre l'autre instable ou inopérante.

- ✚ **La pollution globale** : Chaque outil installé laisse des traces permanentes. À terme, le système hôte devient "lourd", encombré de dépendances obsolètes qui augmentent la surface d'attaque pour les vulnérabilités informatiques.

➤ **L'opacité des interdépendances : Une architecture « boîte noire »**

Le déploiement classique manque cruellement de visibilité sur la topologie de l'application. Au sein d'une même machine, les frontières entre la logique métier (code source) et les services de données (base de données, cache) sont floues.

- ✚ **Absence de cloisonnement** : Une fuite de mémoire dans la logique métier peut saturer les ressources de la base de données, car les deux processus se partagent la même mémoire vive sans limites strictes.
- ✚ **Dépendances implicites** : Contrairement aux recommandations de **Adam Wiggins** dans le manifeste *The Twelve-Factor App*, les liens entre les services sont souvent codés en "dur" (adresses IP locales, chemins de fichiers absolus). Cela rend toute maintenance ou migration extrêmement périlleuse, car l'administrateur n'a pas de vue d'ensemble sur le réseau interne de l'application.

- **Problèmes de portabilité entre environnements (développement, test, production)**
- **Scalabilité coûteuse (ajout de ressources matérielles)**
- **Temps de déploiement longs pour les nouvelles versions**
- **Le problème d'administration**
- **Problème d'optimisation des ressources utilisés.**

Cette problématique justifie le développement d'un outil d'aide à la migration tout en minimisant les risques opérationnels.

III. QUESTIONS DE RECHERCHE

- ✓ Comment planifier une migration progressive sans rupture de service, tout en garantissant la performance, la sécurité et la maintenabilité ?
- ✓ Comment adapter une application web existante monolithique/3-tiers pour les environnements containerisés ?
- ✓ Comment remédier aux problèmes de déploiement ?
- ✓ Quels sont les principes conceptuels et les architectures sous-jacentes pour la migration automatisée d'applications web vers un environnement conteneurisé ?
- ✓ Comment cet outil identifie et gère-t-il les dépendances, les configurations et la persistance des données des applications web existantes pour la conteneurisation ?
- ✓ Quels sont les bénéfices et les limitations inhérents à l'approche automatisée de la migration, notamment en termes de performance, de sécurité et d'effort de refactorisation ?
- ✓ Comment la complexité d'une application web existante influence-t-elle la capacité et l'efficacité d'un outil d'aide automatisé à la conteneuriser ?

IV. OBJECTIFS

L'objectif principal est de permettre à toute structure souhaitant faire une migration d'une application web existante vers un environnement conteneurisé sans exiger de compétences techniques avancées.

Les objectifs spécifiques sont les suivants:

- ✓ Réduire le temps et la complexité de la containerisation d'une application web ;
- ✓ Réduire le temps de déploiement et de redéploiement ;
- ✓ Résoudre le problème de portabilité entre les environnements ;
- ✓ Résoudre le problème de cohabitation des applications sur le même serveur ;

- ✓ Optimiser les processus de migration en réduisant les erreurs humaines, en accélérant les délais et en libérant des ressources humaines pour des tâches à plus forte valeur ajoutée ;
- ✓ Optimiser l'utilisation des ressources ;
- ✓ Permettre une transition fluide vers des orchestrateurs comme Kubernetes ou Docker-swarm pour pallier au problème de montée en charge.

V. PROBLEMES OBSERVES ET LES HYPOTHESES DE L'ETUDE

La migration d'applications web existantes vers un environnement conteneurisé, bien que prometteuse, ne s'effectue pas sans son lot de défis. Selon, les résultats des questionnaires par rapport au paysage de l'écosystème camerounais et les retours d'expérience de l'industrie déploiement, nous formulons ci-dessous les problèmes majeur observés ainsi que les hypothèses de gains attendus durant un tel projet de migration.

➤ Problèmes Observés

Malgré les avantages, la migration d'applications web existantes vers un environnement conteneurisé est confrontée à des problèmes et défis non négligeables, notamment dans le contexte camerounais :

- **P1 : Complexité de la gestion des dépendances et des services externes.** Les applications web existantes possèdent souvent des dépendances complexes (bases de données, caches, services tiers) qui ne sont pas toujours conçues pour un environnement éphémère et distribué. La « **conteneurisation** » de la base de données ou la transition vers des services managés posera des défis de **persistance**, de **configuration** et de **sécurité**.
- **P2 : Gestion de la persistance des données.** Par nature, les conteneurs sont éphémères. Assurer la persistance des données (fichiers uploadés, configurations, bases de données) de manière fiable et sécurisée dans un environnement conteneurisé, sans perte ni corruption, représente un défi majeur. Les solutions de volumes persistants et de stockage partagé nécessitent une planification rigoureuse.

- **P3 : Adaptation des configurations et des paramètres spécifiques à l'Environnement.** Les applications « **legacy** » ont souvent des configurations « **hardcodées** » ou dépendantes de l'environnement hôte. La transition vers des configurations basées sur des variables d'environnement, des fichiers de configuration externes ou des « **secrets** » Kubernetes requiert des modifications significatives et une compréhension approfondie des pratiques « **cloud-native** ».
- **P4 : Défis liés à la sécurité des conteneurs et de l'orchestration.** La conteneurisation introduit de nouvelles surfaces d'attaque (vulnérabilités dans les images de base, mauvaise configuration des conteneurs, problèmes d'isolation, gestion des accès et des secrets). Assurer une sécurité robuste nécessite des compétences spécifiques et l'implémentation de bonnes pratiques à chaque étape du cycle de vie.
- **P5 : Courbe d'apprentissage et manque de compétences locales.** L'adoption de technologies comme Docker et Kubernetes requiert une montée en compétences des équipes de développement et d'opérations. Au Cameroun, le manque d'experts expérimentés dans ces technologies peut ralentir le processus de migration et augmenter les coûts de formation.
- **Hypothèses (Gains Attendus)**

La migration d'applications web vers un environnement conteneurisé permettra d'atteindre les gains significatifs suivants :

- **H1 : Amélioration de la portabilité et de la cohérence des environnements.** La conteneurisation encapsulera les applications et leurs dépendances, garantissant une exécution identique et fiable entre les environnements de développement, de test et de production, réduisant ainsi les erreurs liées aux « **différences d'environnement** ».
- **H2 : Optimisation de l'utilisation des ressources matérielles.** Grâce à la légèreté des conteneurs par rapport aux machines virtuelles, une meilleure densité d'applications sur les serveurs physiques ou virtuels existants sera observée, entraînant une réduction potentielle des coûts d'infrastructure et une meilleure exploitation des ressources disponibles, un point crucial dans des contextes aux infrastructures parfois limitées.

- **H3 : Accélération des cycles de déploiement et simplification de la gestion.** L'intégration des conteneurs dans des pipelines d'intégration et de déploiement continus (CI/CD) permettra des mises à jour plus fréquentes, plus rapides et moins risquées, améliorant l'agilité et la réactivité des équipes.
- **H4 : Amélioration de la scalabilité et de la résilience des applications.** Les applications conteneurisées, orchestrées par des plateformes comme Kubernetes, pourront facilement monter ou descendre en charge ("scaler") horizontalement pour s'adapter aux variations de trafic, et bénéficieront de mécanismes d'auto-guérison en cas de défaillance.
- **H5 : Réduction des Coûts opérationnels à long terme.** Bien qu'un investissement initial soit nécessaire, la standardisation, l'automatisation et l'optimisation des ressources offertes par la conteneurisation mèneront à une réduction des coûts de maintenance, d'exploitation et d'infrastructure sur le long terme.

Ces hypothèses et problèmes serviront de fil conducteur à notre recherche, permettant d'analyser non seulement les bénéfices potentiels de la migration, mais aussi les obstacles concrets qui devront être adressés pour une transition réussie, en particulier dans le contexte spécifique du Cameroun.

VI. INTERET DE L'ETUDE

L'étude de la migration des applications web vers un environnement conteneurisé est déjà un sujet d'un grand intérêt, comme nous l'avons souligné. Cependant, l'ajout de la dimension d'un « **outil d'aide automatisé** » confère à cette étude une valeur ajoutée significative et un intérêt encore plus marqué, en se positionnant à l'intersection de la modernisation des infrastructures et de l'optimisation des processus.

Cet intérêt peut être articulé autour de plusieurs axes, consolidant les bénéfices déjà identifiés tout en introduisant de nouveaux concepts liés à l'automatisation.

➤ Intérêt technologique et méthodologique

L'intégration d'un outil d'aide automatisé place cette étude au cœur des avancées en matière de **DevOps** et de **transformation numérique industrialisée**. Elle permettra de :

- ✓ **Démystifier et simplifier la migration** : Un outil automatisé peut abstraire une grande partie de la complexité technique inhérente à la conteneurisation et à l'orchestration. L'étude visera à comprendre comment un tel outil peut simplifier des tâches complexes telles que la détection des dépendances, la génération de Dockerfiles, Docker-compose, la configuration des manifestes Kubernetes, ou même l'adaptation de code.
- ✓ **Accélérer le processus de migration** : L'automatisation est synonyme de rapidité. L'étude quantifiera comment un outil peut réduire considérablement le temps nécessaire à la migration, en minimisant les interventions manuelles et en éliminant les tâches répétitives, souvent sources d'erreurs.
- ✓ **Réduire les erreurs humaines** : Les processus manuels sont sujets aux erreurs. L'analyse de l'impact d'un outil automatisé sur la réduction des erreurs de configuration, de déploiement ou de dépendances est un point clé.
- ✓ **Standardiser les bonnes pratiques** : Un outil d'aide peut embarquer et appliquer automatiquement les meilleures pratiques en matière de sécurité (images minimales, utilisateurs non-root), de performance (optimisation des couches d'images) et de résilience (probes de santé Kubernetes). L'étude explorera comment l'outil impose ou facilite l'adoption de ces standards.
- ✓ **Améliorer la reproductibilité et la Cohérence** : Garantir que les migrations sont effectuées de manière cohérente d'une application à l'autre ou d'un environnement à l'autre est crucial. L'automatisation assure cette reproductibilité, et l'étude évaluera l'efficacité de l'outil dans cet objectif.

- ✓ **Faire le Pont entre Legacy et Cloud-Native** : L'outil d'aide automatisé agit comme un facilitateur, aidant les applications legacy à s'adapter aux exigences des environnements cloud-native sans nécessiter une réécriture complète immédiate.
- ✓ Assurer le déploiement automatique des outils d'administrations tels que **portainer**, **Grafana** et **prometheus** pour se rassurer de la santé de l'application

➤ **Intérêt économique et Opérationnel**

L'ajout de l'automatisation amplifie les bénéfices économiques et opérationnels de la migration :

- ✓ **Optimisation des Ressources Humaines** : En automatisant les tâches répétitives et complexes, les équipes techniques peuvent se concentrer sur des activités à plus forte valeur ajoutée (architecture, innovation, optimisation). L'étude évaluera l'efficacité de l'outil à libérer du temps pour les experts.
- ✓ **Réduction des Coûts Globaux de Migration** : Au-delà des économies d'infrastructure, l'automatisation permet de réduire les coûts liés aux heures de travail, à la formation intensive pour des tâches spécifiques et à la correction d'erreurs post-migration.
- ✓ **Scalabilité des Opérations de Migration** : Un outil automatisé permet de gérer la migration d'un grand nombre d'applications avec une efficacité accrue, ce qui est essentiel pour les organisations disposant d'un vaste patrimoine applicatif.
- ✓ **Démocratisation de l'Accès à la Conteneurisation** : En simplifiant le processus, un tel outil peut rendre la conteneurisation accessible à des équipes ou des entreprises qui manquent de l'expertise pointue nécessaire pour une migration entièrement manuelle.

➤ **Intérêt Stratégique et Pédagogique**

Cet axe d'étude est d'une grande pertinence pour l'avenir de la gestion des infrastructures et du développement logiciel :

- ✓ **Avantage Concurrentiel** : Démontrer comment un outil automatisé peut conférer un avantage concurrentiel aux entreprises, leur permettant de moderniser leur stack technologique plus rapidement et à moindre coût que leurs concurrents.
- ✓ **Feuille de Route pour l'Adoption de l'IA/ML en DevOps** : L'étude peut ouvrir la voie à des recherches futures sur l'intégration de l'intelligence artificielle et du machine learning dans les outils d'automatisation de migration, permettant des analyses plus fines et des optimisations prédictives.
- ✓ **Préparation aux Architectures Futures** : La maîtrise de l'automatisation de la migration est une étape clé vers l'adoption d'architectures cloud-native complètes, y compris les microservices et le serverless, en préparant les systèmes et les équipes à des évolutions futures.

En intégrant la dimension d'un outil d'aide automatisé, cette étude ne se contente plus d'analyser comment migrer, mais se concentre sur **comment optimiser et industrialiser** ce processus de migration. Cela en fait un sujet d'une grande valeur pratique et théorique, susceptible d'apporter des contributions significatives à la recherche et au développement dans le domaine de l'ingénierie logicielle et des opérations informatiques modernes.

VII. APPROCHE METHODOLOGIQUE

Cette section délimite la démarche méthodologique adoptée pour investiguer et évaluer la migration d'applications web vers un environnement conteneurisé, en mettant un accent particulier sur **l'utilisation et l'impact d'un outil d'aide automatisé**. L'approche combinera l'expérimentation pratique via une étude de cas, l'évaluation des performances et l'analyse qualitative des processus, le tout dans le contexte des défis et opportunités spécifiques au Cameroun.

➤ **Approche de recherche choisie : Étude de cas comparative et expérimentation**

Notre recherche s'articulera autour d'une **approche hybride et comparative**, centrée sur une **étude de cas pratique**. L'objectif est de non seulement réaliser une migration, mais aussi d'évaluer l'efficacité et les bénéfices d'un outil d'automatisation dans ce processus.

a) Étude de Cas Pratique et Contextualisée

Nous sélectionnerons une **application web existante**, idéalement représentative du patrimoine applicatif camerounais (par exemple, une application d'entreprise ou administrative, développée avec des technologies courantes comme **PHP/Laravel, Python/Django, Node.js/Express, ou Java/Spring Boot**). Cette application servira de cobaye pour la migration. La sélection tiendra compte de sa complexité (nombre de dépendances, services externes, persistance de données) afin de mettre en lumière les défis réels et les capacités de l'outil d'automatisation.

b) Approche Comparative

La migration de l'application sera effectuée selon **deux scénarios distincts** pour permettre une évaluation comparative de l'outil d'automatisation :

- **Migration Manuelle (Scénario de Référence)** : Les étapes de conteneurisation (création des Dockerfiles, configuration de Docker Compose) et de déploiement sur l'orchestrateur (manifestes Kubernetes) seront réalisées manuellement ou avec des scripts basiques. Ce scénario établira une base de référence en termes de temps, d'efforts et de complexité perçue.
- **Migration Assistée par un Outil d'Automatisation** : La même application sera migrée en utilisant un **outil d'aide automatisé** conçu pour faciliter la conteneurisation et la génération des artefacts de déploiement. Ce scénario permettra d'évaluer l'impact direct de l'outil sur le processus.

c) Évaluation Quantitative

Des mesures précises seront effectuées pour quantifier les gains ou les défis de chaque scénario. Cela inclura :

- **Temps de migration** : Mesure du temps nécessaire pour conteneuriser et déployer l'application dans les deux scénarios.
- **Effort de configuration** : Quantité de code ou de configuration manuel requis.
- **Performance de l'application** : Mesure des temps de réponse, du débit et de l'utilisation des ressources (CPU, RAM) de l'application conteneurisée par rapport à son état initial non conteneurisé, ainsi qu'une comparaison entre les deux scénarios de migration.

d) Analyse Qualitative

Une analyse qualitative viendra compléter les données chiffrées, en documentant :

- La **facilité d'utilisation** de l'outil automatisé.
- La **qualité et l'optimisation** des artefacts générés par l'outil (Dockerfiles, manifestes Kubernetes).
- Les **défis spécifiques** rencontrés avec l'outil ou lors de la migration manuelle.
- Les **compétences requises** pour chaque approche.
- Les **bonnes pratiques** identifiées.

➤ Outils et Technologies Utilisés

La réalisation de cette étude nécessitera un ensemble d'outils et de technologies rigoureusement choisis pour simuler un environnement de production réaliste et évaluer l'outil d'automatisation.

a) Application Web Cible :

- ✓ Une application web existante (ex: un système de gestion la comptabilité, un portail e-commerce simple, ou une application de gestion des equivalences au cameroun). Les technologies sous-jacentes (ex: PHP/MySQL, Python/PostgreSQL, Node.js/MongoDB) seront déterminées lors de la phase de préparation ;
- ✓ Un système de gestion de base de données (SGBD) tel que MySQL ou PostgreSQL pour la persistance des données ;

b) Outils de Conteneurisation, d'Orchestration et d'administration :

- ✓ **Docker Engine** : Pour la création des images conteneurs (via Dockerfiles) et la gestion des conteneurs individuels.
- ✓ **Docker Compose** : Pour l'orchestration multi-conteneurs en environnement de développement et pour la comparaison avec les déploiements Kubernetes.
- ✓ **Prometheus** : elle permet de récupérer des métriques de votre application et envoyer à Grafana ;
- ✓ **Grafana** : Le monitoring, ou supervision, intervient une fois que notre application est déployée sur un environnement, que ce soit un environnement de staging, de test, de démonstration ou bien l'environnement de production lui-même.
- ✓ **Portainer** : permet de gérer simplement des conteneurs Docker dans des environnements distribués.

➤ Outil d'Aide à la Migration Automatisée :

C'est l'élément central de cette étude. Nous allons utiliser le langage « Java » qui permet d'écrire des applications modernes de bureau en utilisant les technologies javaFx. Le langage Java est un choix incontournable pour le développement d'applications grâce à sa portabilité « écrire une fois, exécuter partout » (JVM), sa robustesse, sa sécurité renforcée, et son orientation objet qui facilite la maintenance. Il est idéal pour les applications d'entreprise, le Big Data, et Android.

Voici les principaux avantages de Java pour la création d'applications :

- ✓ **Indépendance de la plateforme (Portabilité)** : Le bytecode Java peut s'exécuter sur n'importe quel système d'exploitation (Windows, Linux, macOS) doté d'une machine virtuelle Java (JVM).

- ✓ **Écosystème riche et mature** : Une vaste gamme de bibliothèques, de frameworks (Spring, Hibernate) et d'outils (Maven, Gradle) accélère le développement et la standardisation.
- ✓ **Orientation Objet (POO)** : Java permet de créer des applications modulaires, flexibles et réutilisables.
- ✓ **Sécurité et Robustesse** : Java offre une gestion automatique de la mémoire (garbage collection) et des mécanismes de sécurité intégrés (sandboxing, vérification de bytecode) pour protéger les données.
- ✓ **Performance et Évolutivité** : Grâce au multithreading et à la compilation JIT (Just-In-Time), Java est performant pour les applications lourdes et les systèmes d'entreprise.
- ✓ **Communauté importante** : Une vaste communauté de développeurs assure une documentation abondante et une résolution rapide des problèmes.

VIII. SCOPE ET LIMITES DE L'ÉTUDE

Cette section définit précisément le périmètre d'action de cette étude, clarifiant ce qu'elle couvrira et, de manière tout aussi importante, ce qu'elle ne couvrira pas. La spécificité d'explorer la migration à travers un outil d'aide automatisé introduit des contraintes et des focus particuliers.

➤ **Scope de l'Étude**

Le scope de cette étude est défini par les éléments clés suivants :

- a) **Cœur de la Migration** : L'étude se concentrera sur le processus technique de migration d'applications web existantes d'un environnement traditionnel (serveur dédié, machine virtuelle) vers un environnement conteneurisé. Cela inclut :
 - ✓ L'analyse des architectures d'applications web monolithiques ou traditionnelles (par exemple, Xampp, applications basées sur des frameworks PHP, Python, Java Servlets/JSP).

- ✓ La création d'images conteneurs (Dockerfiles) optimisées pour ces applications, en respectant les bonnes pratiques (légèreté, sécurité, externalisation de la configuration).
- ✓ La gestion de la **persistance des données** (bases de données, fichiers uploads) dans un contexte conteneurisé.

b) Outil d'Aide Automatisé : L'étude explorera la faisabilité et l'impact de l'utilisation (ou la conception conceptuelle des fonctionnalités clés) d'un **outil d'aide automatisé** dans le processus de migration. Cela impliquera :

- ✓ **Analyse des outils existants :** Un examen des outils open-source ou commerciaux qui proposent des fonctionnalités d'automatisation pour la migration (e.g., outils de scan d'applications pour la conteneurisation, générateurs de Dockerfiles/manifestes Kubernetes, outils de modernisation).
- ✓ **Proposition d'un cadre d'automatisation :** S'il n'existe pas d'outil pleinement adapté ou accessible dans le contexte donné, l'étude proposera un **cadre conceptuel** ou une **liste de fonctionnalités clés** qu'un tel outil devrait offrir pour faciliter la migration dans le contexte camerounais.
- ✓ **Démonstration de l'automatisation :** À travers l'étude de cas, nous tenterons de mettre en œuvre des aspects de cette automatisation (par exemple, script de génération de Dockerfile basique, utilisation d'un convertisseur de docker-compose en k8s manifests, ou un script pour l'analyse des dépendances).

c) Contexte Camerounais : L'étude intégrera les spécificités du contexte technologique camerounais en tenant compte de :

- ✓ Les infrastructures disponibles.
- ✓ Les compétences techniques locales.
- ✓ Les défis potentiels liés à la connectivité ou aux ressources.

- ✓ L'objectif est que les recommandations et les leçons apprises soient directement pertinentes pour les organisations opérant au Cameroun.

d) Mesure de l'Impact : L'étude inclura la **mesure de la performance** (temps de réponse, consommation de ressources) de l'application avant et après migration, pour quantifier les bénéfices apportés par la conteneurisation et l'automatisation.

➤ **Limites de l'Étude**

Afin de maintenir la faisabilité et la profondeur de l'analyse dans le cadre d'un mémoire de Master, plusieurs limites ont été délibérément établies :

a) Nature de l'Outil d'Aide Automatisé :

- L'étude ne vise à **développer un outil d'aide automatisé et prêt à la production pour la migration.**
- La génération initiale de Dockerfiles, la conversion de configurations.

b) Type d'Applications Web

L'étude se concentrera sur la migration d'applications web « **standard** » basées sur des technologies et architectures courantes (ex: applications web dynamiques avec base de données). Elle n'abordera **pas** spécifiquement :

- Les applications web à très haute criticité nécessitant des certifications spécifiques (e.g., bancaires ultra-réglémentées).
- Les applications basées sur des technologies obsolètes et très complexes à conteneuriser sans refactoring majeur.
- Les applications nécessitant des drivers matériels spécifiques ou une interaction très bas niveau avec le système d'exploitation hôte.
- Les architectures de microservices natives, la migration étant ici vue comme un processus de modernisation d'applications existantes, et non la construction de nouvelles architectures.

c) Profondeur de l'Optimisation Post-Migration

Bien que la performance sera mesurée, l'étude ne se penchera pas sur des optimisations très avancées spécifiques à Kubernetes (ex: optimisation fine des **resource limits**, **request** des pods, **network policies** très complexes, stratégies de scalabilité horizontale très sophistiquées comme les **Horizontal Pod Autoscalers** basés sur des métriques personnalisées), sauf si cela est directement lié au fonctionnement de l'outil d'aide.

Les aspects de **sécurité** seront abordés sous l'angle des bonnes pratiques de conteneurisation (images, utilisateurs non-root), mais l'étude ne mènera pas une analyse de pénétration exhaustive ou une implémentation de politiques de sécurité de cluster avancées.

d) Considérations humaines et organisationnelles

L'étude se focalisera principalement sur les aspects **techniques et outillés** de la migration. Les défis liés au changement de culture DevOps, à la formation des équipes, à la gestion du changement organisationnel, ou à la gouvernance IT ne seront pas le cœur de l'analyse, bien qu'ils seront reconnus comme des facteurs critiques dans la réussite globale d'une migration.

e) Coût et Rentabilité

Bien que l'intérêt économique soit mentionné, l'étude ne réalisera pas une analyse financière ou un calcul de Retour sur Investissement (ROI) détaillé de la migration dans un contexte précis d'entreprise camerounaise. Elle se limitera à une discussion qualitative des bénéfices économiques.

Ces limites ont été définies pour garantir la faisabilité du projet dans le temps imparti pour un mémoire de Master, tout en permettant une exploration approfondie et rigoureuse des problématiques centrales.